

Graph Problems and Solutions

1. Number of Islands

Problem:

Given a 2D grid of 1s (land) and 0s (water), count the number of islands. An island is a group of connected lands (horizontally or vertically).

Dynamic Programming Approach:

- We can use a Union-Find (Disjoint Set Union) approach, which is often used for solving grid connectivity problems.
- Alternatively, we can treat the grid as a DP table and mark cells as visited when we discover an island

Pseudocode (Union-Find):

```
class UnionFind {
```

```
public:
```

```
vector parent;
```

```
int count;
```

```
UnionFind(int n) {
```

```
parent = vector(n, -1);
```

```
count =
```

```
0;
```

```
}
```

```
int find(int x) {
```

```
if (parent[x] == -1) return x;
```

```
return parent[x] = find(parent[x]);
```

```
}
```

```
void unionSets(int x, int y) {
```

```
int rootX = find(x);
```

```
int rootY = find(y);
```

```
if (rootX != rootY) {
```

```
parent[rootX] = rootY;
```

```
count--;
```

```
}
```

```
}
```

```
};
```

```
int numIslands(vector<vector<char>>& grid) {
```

```
int m = grid.size(), n = grid[0].size();
```

```
UnionFind uf(m * n);
```

```
for (int i = 0; i < m; i++) {
```

```
for (int j = 0; j < n; j++) {
```

```
if (grid[i][j] == '1') {
```

```
uf.count++;
```

```
if (i > 0 && grid[i-1][j] ==
```

```

'1')
uf.unionSets(i * n + j, (i-1) * n + j);
if (j > 0 && grid[i][j-1] == '1')
uf.unionSets(i * n + j, i * n + j - 1);
}
}
}
return uf.count;
}

```

- Time Complexity: $O(m * n)$
- Space Complexity: $O(m * n)$

2. Max Area of Island

Problem:

Given a 2D grid of 1s (land) and 0s (water), find the maximum area of an island (group of connected lands).

Dynamic Programming Approach:

- Use Depth First Search (DFS) or Breadth First Search (BFS) to explore the connected land area and calculate the size.
- Maintain a DP table to store the results of already visited cells to avoid recomputation.

Pseudocode

(DFS):

```
int dfs(vector>& grid, int i, int j) {  
    if (i < 0 || j < 0 || i >= grid.size() || j >= grid[0].size() || grid[i][j] == 0)  
        return 0;  
    grid[i][j] = 0; //Mark the cell as visited  
    int area = 1;  
    area += dfs(grid, i + 1, j);  
    area += dfs(grid, i - 1, j);  
    area += dfs(grid, i, j + 1);  
    area += dfs(grid, i, j - 1);  
    return area;  
}
```

```
int maxAreaOfIsland(vector>& grid) {  
    int max_area = 0;  
    for (int i = 0; i < grid.size(); i++) {  
        for (int j = 0; j < grid[0].size(); j++) {  
            if (grid[i][j] == 1) {  
                max_area = max(max_area, dfs(grid, i, j));  
            }  
        }  
    }  
    return
```

```
max_area;
```

```
}
```

- Time Complexity: $O(m * n)$
- Space Complexity: $O(m * n)$ due to recursion stack.

3. Clone Graph

Problem:

Given a reference to a node in a graph, clone the entire graph.

Dynamic Programming Approach:

- Use DFS with memoization to clone nodes and their neighbors.
- For each node, cache the result of the cloned node to avoid re-cloning.

Pseudocode (DFS with Memoization):

```
unordered_map clone_map;
```

```
Node* dfs(Node* node) {
```

```
    if (!node) return nullptr;
```

```
    if (clone_map.find(node) != clone_map.end()) return clone_map[node];
```

```
    Node* clone_node = new Node(node->val);
```

```
    clone_map[node] = clone_node;
```

```
    for (Node* neighbor : node->neighbors)
```

```

{
clone_node->neighbors.push_back(dfs(neighbor));
}
return clone_node;
}

```

```

Node* cloneGraph(Node* node) {
return dfs(node);
}

```

- Time Complexity: $O(V + E)$
- Space Complexity: $O(V)$

4. Walls and Gates

Problem:

You are given a grid of rooms, where each room can be a wall (-1), a gate (0), or an empty room (INF). Fill each empty room with the distance to its nearest gate.

Dynamic Programming Approach:

- Perform a multi-source BFS to propagate the minimum distance to all rooms. Initially, mark all gates with distance 0, and then for each empty room, propagate the distance to its nearest gate.
- Use a DP array to store

distances.

Pseudocode (Multi-source BFS):

```
void wallsAndGates(vector>& rooms) {
```

```
    queue> q;
```

```
    int rows = rooms.size(), cols = rooms[0].size();
```

```
    // Add all gates to the queue
```

```
    for (int i = 0; i < rows; i++) {
```

```
        for (int j = 0; j < cols; j++) {
```

```
            if (rooms[i][j] == 0) q.push({i, j});
```

```
        }
```

```
    }
```

```
    // Perform BFS
```

```
    while (!q.empty()) {
```

```
        auto [x, y] = q.front();
```

```
        q.pop();
```

```
        for (auto dir : {{0,1}, {1,0}, {0,-1}, {-1,0}}) {
```

```
            int new_x = x + dir[0], new_y = y + dir[1];
```

```
            if (new_x >= 0 && new_y >= 0 && new_x < rows && new_y < cols &&
```

```
                rooms[new_x][new_y] == INF) {
```

```
                rooms[new_x][new_y] = rooms[x][y] + 1;
```

```
                q.push({new_x,
```

```
new_y});
```

```
}
```

```
}
```

```
}
```

```
}
```

- Time Complexity: $O(m * n)$

- Space Complexity: $O(m * n)$

5. Rotting Oranges

Problem:

Given a grid of oranges, where 1 is a fresh orange, 0 is an empty cell, and 2 is a rotten orange, return the minimum time required for all oranges to become rotten. If it's impossible, return -1.

Dynamic Programming Approach:

- Use multi-source BFS starting from all rotten oranges at the beginning and propagate the time it takes for each fresh orange to rot.
- Track the number of steps required to make all oranges rotten.

Pseudocode (Multi-source BFS):

```
int orangesRotting(vector<vector<int>>& grid) {
```

```
    int rows = grid.size(), cols = grid[0].size();
```

```
    queue<int, int> q;
```

```
    int fresh_oranges = 0, time =
```

0;

```
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (grid[i][j] == 2) q.push({i, j});
        if (grid[i][j] == 1) fresh_oranges++;
    }
}

if (fresh_oranges == 0) return 0;
while (!q.empty()) {
    int size = q.size();
    bool rot_occurred = false;
    for (int i = 0; i < size; i++) {
        auto [x, y] = q.front();
        q.pop();
        for (auto dir : {{0, 1}, {1, 0}, {0, -1}, {-1, 0}}) {
            int new_x = x + dir[0], new_y = y + dir[1];
            if (new_x >= 0 && new_y >= 0 && new_x < rows && new_y < cols &&
                grid[new_x][new_y] == 1) {
                grid[new_x][new_y] = 2;
                q.push({new_x, new_y});
                fresh_oranges--;
                rot_occurred =
            }
```

```

true;
}
}
}
if (rot_occurred) time++;
}
return fresh_oranges == 0 ? time : -1;
}

```

- Time Complexity: $O(m * n)$
- Space Complexity: $O(m * n)$

6. Pacific Atlantic Water Flow

Problem:

Given a matrix of integers representing heights, water can flow from any cell to the Pacific and Atlantic oceans. You need to find the cells from where water can flow to both oceans.

Dynamic Programming Approach:

- Use two DP arrays: one for the Pacific Ocean and one for the Atlantic Ocean.
- Start by marking the cells that can flow to the Pacific and Atlantic from the borders. Then propagate the water flow inward to other cells using a DFS approach.

Pseudocode (DP +

DFS):

```
vector> pacificAtlantic(vector>& matrix) {  
    int m = matrix.size(), n = matrix[0].size();  
    vector> pacific(m, vector(n, false)), atlantic(m, vector(n, false));  
    // DFS function to mark the reachable cells  
    function>&> dfs = [&](int i, int j, vector>& ocean) {  
        ocean[i][j] = true;  
        for (auto dir {0, 1}, {1, 0}, {0, -1}, {-1, 0}) {  
            int ni = i + dir[0], nj = j + dir[1];  
            if (ni >= 0 && nj >= 0 && ni < m && nj < n && ocean[ni][nj] &&  
                matrix[ni][nj] >= matrix[i][j]) {  
                dfs(ni, nj, ocean);  
            }  
        }  
    };  
    // DFS from Pacific Ocean (top and left edges)  
    for (int i = 0; i < m; i++) dfs(i, 0, pacific);  
    for (int j = 0; j < n; j++) dfs(0, j, pacific);  
    // DFS from Atlantic Ocean (bottom and right edges)  
    for (int i = 0; i < m; i++) dfs(i, n - 1, atlantic);  
    for (int j = 0; j < n; j++) dfs(m - 1, j, atlantic);  
  
    // Find all cells that can reach both oceans  
    vector> result;  
    for (int i = 0; i < m; i++)
```

```

{
for (int j = 0; j < n; j++) {
if (pacific[i][j] && atlantic[i][j]) result.push_back({i, j});
}
}
return result;
}

```

- Time Complexity: $O(m * n)$
- Space Complexity: $O(m * n)$

9. Surrounded Regions

Problem:

Given a 2D board filled with 'X' and 'O', capture all regions surrounded by 'X'. The regions can only be surrounded if all 'O' cells are completely enclosed by 'X'.

Dynamic Programming Approach:

- Use DFS or BFS to mark all 'O' cells connected to the border (since they can't be surrounded).
- After marking, traverse the board to change all 'O' to 'X' and 'X' to 'O' where necessary.

Pseudocode (DFS Approach):

```

void dfs(vector>& board, int i, int j) {
if (i < 0 || j < 0 || i >= board.size() || j >= board[0].size() || board[i][j]

```

```

!= '0') return;
board[i][j] = '#'; //Mark as visited
dfs(board, i + 1, j);
dfs(board, i - 1, j);
dfs(board, i, j + 1);
dfs(board, i, j - 1);
}

void solve(vector<vector>& board) {
if (board.empty()) return;
int m = board.size(), n = board[0].size();

// Run DFS from all border '0's
for (int i = 0; i < m; i++) {
if (board[i][0] == '0') dfs(board, i, 0);
if (board[i][n - 1] == '0') dfs(board, i, n - 1);
}

for (int j = 0; j < n; j++) {
if (board[0][j] == '0') dfs(board, 0, j);
if (board[m - 1][j] == '0') dfs(board, m - 1, j);
}

// Change remaining '0' to 'X' and '#' back to '0'
for (int i = 0; i < m; i++) {
for (int j = 0; j < n; j++) {
if (board[i][j] == '0') board[i][j] = 'X';
else if (board[i][j] == '#') board[i][j] =

```

'0';

}

}

}

- Time Complexity: $O(m * n)$

- Space Complexity: $O(m * n)$

10. Course Schedule

Problem:

Given a list of prerequisites for a set of courses, determine if it is possible to finish all the courses.

Dynamic Programming Approach:

- Treat this problem as a graph problem where each course is a node and prerequisites are directed edges.
- Use Topological Sorting and check for cycles. If a cycle exists, it's impossible to finish all courses.

Pseudocode (Topological Sort using DFS):

```
bool dfs(int course, vector<int>& graph, vector<int>& visited, vector<int>& result) {
```

```
    if (visited[course] == 1) return false; //cycle detected
```

```
    if (visited[course] == 2) return true; //already visited
```

```
    visited[course] = 1;
```

```
    for (int neighbor : graph[course])
```

```

{
    if (!dfs(neighbor, graph, visited, result)) return false;
}
visited[course] = 2;
result.push_back(course);
return true;
}

bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
    vector<vector<int>> graph(numCourses);
    vector<int> visited(numCourses, 0);
    vector<int> result;

    for (auto& prereq : prerequisites) {
        graph[prereq[1]].push_back(prereq[0]);
    }

    for (int i = 0; i < numCourses; i++) {
        if (visited[i] == 0 && !dfs(i, graph, visited, result)) {
            return false;
        }
    }

    return true;
}

```

- Time Complexity: $O(V + E)$
- Space Complexity: $O(V +$

E)

11. Course Schedule II

Problem:

Given a list of prerequisites for a set of courses, return the order in which courses should be taken. If it's not possible to finish all courses, return an empty list.

Dynamic Programming Approach:

- Similar to Course Schedule I, we use Topological Sorting to determine the order of courses.

Pseudocode (Topological Sort):

```
vector findOrder(int numCourses, vector<vector<int>>& prerequisites) {  
    vector<vector<int>> graph(numCourses);  
    vector<int> indegree(numCourses, 0);  
    vector<int> result;  
    // Build graph and calculate in-degrees  
    for (auto& prereq : prerequisites) {  
        graph[prereq[1]].push_back(prereq[0]);  
        indegree[prereq[0]]++;  
    }  
    queue<int> q;  
    for (int i = 0; i < numCourses; i++)
```

```

{
if (indegree[i] == 0) q.push(i);
}
while (!q.empty()) {
int course = q.front();
q.pop();
result.push_back(course);

for (int neighbor : graph[course]) {
if (--indegree[neighbor] == 0) {
q.push(neighbor);
}
}
}
return result.size() == numCourses ? result : vector();
}

```

- Time Complexity: $O(V + E)$
- Space Complexity: $O(V + E)$

12. Graph Valid Tree

Problem:

Given an undirected graph with n nodes and m edges, check if the graph is a valid

tree.

Dynamic Programming Approach:

- Use DFS or BFS to check if the graph is connected and has exactly $n-1$ edges.

Pseudocode (DFS):

```
bool dfs(int node, vector<int>& adj, vector<bool>& visited, int parent) {
    visited[node] = true;
    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            if (!dfs(neighbor, adj, visited, node)) return false;
        } else if (neighbor != parent) {
            return false; //A cycle detected
        }
    }
    return true;
}

bool validTree(int n, vector<int>& edges) {
    if (edges.size() != n - 1) return false; //Tree condition
    vector<int> adj(n);
    for (auto& edge : edges)
```

```

{
adj[edge[0]].push_back(edge[1]);
adj[edge[1]].push_back(edge[0]);
}

vector visited(n, false);
if (!dfs(0, adj, visited, -1)) return false;
for (bool v : visited) {
if (!v) return false; //If any node is unvisited, it's disconnected
}
return true;
}

```

- Time Complexity: $O(V + E)$
- Space Complexity: $O(V + E)$

13. Number of Connected Components in an Undirected Graph

Problem:

Given an undirected graph, return the number of connected components.

Dynamic Programming Approach:

- Use DFS or BFS to explore the graph and count the number of connected components.

Pseudocode

(DFS):

```
void dfs(int node, vector<int>& graph, vector<bool>& visited) {  
    visited[node] = true;  
    for (int neighbor : graph[node]) {  
        if (!visited[neighbor]) dfs(neighbor, graph, visited);  
    }  
}
```

```
int countComponents(int n, vector<vector<int>>& edges) {  
    vector<vector<int>> graph(n);  
    for (auto& edge : edges) {  
        graph[edge[0]].push_back(edge[1]);  
        graph[edge[1]].push_back(edge[0]);  
    }  
    vector<bool> visited(n, false);  
    int count = 0;
```

```
    for (int i = 0; i < n; i++) {  
        if (!visited[i]) {  
            dfs(i, graph, visited);  
            count++;  
        }  
    }  
    return count;  
}
```

- Time Complexity: $O(V + E)$

$E)$

- Space Complexity: $O(V + E)$

14. Redundant Connection

Problem:

Given a list of edges in a graph, find the redundant connection.

Dynamic Programming Approach:

- Use Union-Find to detect cycles and identify the redundant connection.

Pseudocode (Union-Find):

```
class UnionFind {
public:
    vector parent;
    UnionFind(int n) {
        parent = vector(n, -1);
    }
    int find(int x) {
        if (parent[x] == -1) return x;
        return parent[x] = find(parent[x]);
    }
    void unionSets(int x, int y) {
        int rootX = find(x);
        int rootY =
```

```

find(y);
if (rootX  $\neq$  rootY) {
    parent[rootX] = rootY;
}
}
};

vector findRedundantConnection(vector<int>& edges) {
    int n = edges.size();
    UnionFind uf(n + 1);
    for (auto& edge : edges) {
        if (uf.find(edge[0]) == uf.find(edge[1])) return edge;
        uf.unionSets(edge[0], edge[1]);
    }
    return {};
}

- Time Complexity:  $O(E * \alpha(V))$ 
- Space Complexity:  $O(V)$ 

```

15. Word Ladder

Problem:

Given a start word, an end word, and a dictionary, find the length of the shortest transformation sequence from start to end. Each transformed word must exist in the